Object Detection with LiDAR Point Cloud and Estimation of 3D Bounding Boxes

Abstract:

The objective is to connect raw LiDAR returns and detect the object in the environment and to estimate the 3D bounding boxes for the object. This project builds a Python pipeline for inspecting KITTI LiDAR point clouds and visualizing labeled objects as oriented 3D boxes. It reads Velodyne binary scans, reshapes each return into x, y, z and reflectance values, applies matching LiDAR to camera and camera to rectification matrices and interprets object dimensions, location, and yaw from KITTI label files. Then the code constructs a cuboid for each supported label, counts transformed LiDAR returns inside each cuboid and displays the point cloud and boxes in Open3D. The currently used training dataset contains 20 frames.

Data used	Description
Input sample	20 KITTI mini frames with matching scans, labels, and calibration
Supported labels	Car, Truck, Pedestrian, Cyclist, Van
Output	Height-colored point-cloud plots, point counts per object, interactive Open3D viewer

Data and Processing workflow:

This project uses the KITTI mini training dataset. Each frame has three matching inputs: a Velodyne scan, a label file and a calibration file. KITTI provides an object-detection benchmark and a development kit.

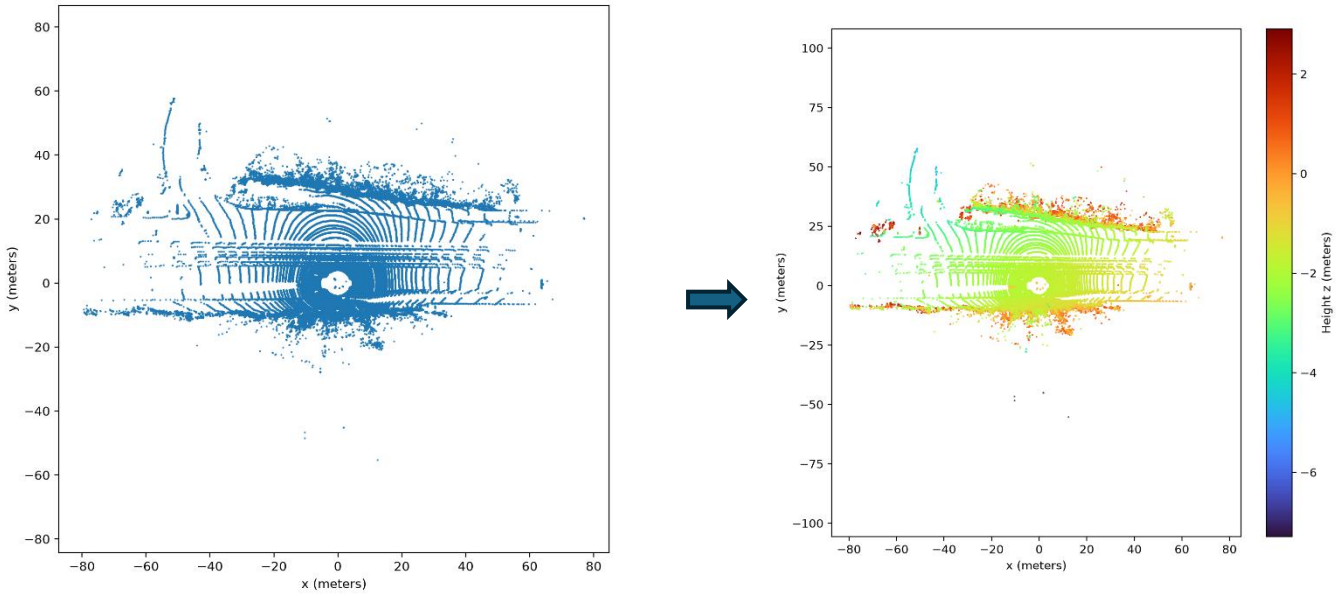
Input	How the code uses it
Velodyne (bin file)	Values are grouped into rows of x, y, z, reflectance; for example, frame 000001 contains 120,268 returns.
Label (txt file)	Object class, 2D fields, 3D dimensions, camera-frame location, and yaw; DontCare rows are excluded.
Calibration (txt file)	Tr_velo_to_cam and R0_rect matrices are used to express LiDAR points in rectified camera coordinates.

Visualization from Point Clouds to 3D Bounding box Geometry

Plotting of Points in 2D viewer

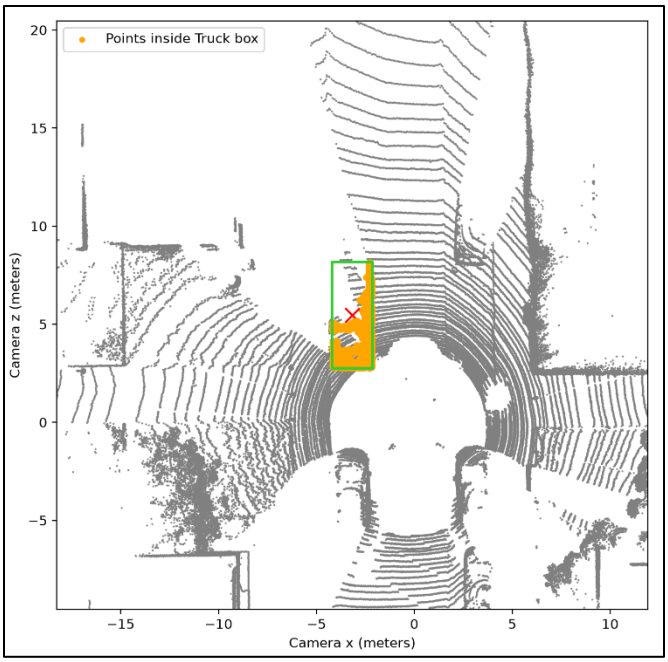
The return points to the LiDAR are obtained from the Velodyne file are plotted in a viewer, then the object class, dimensions, location and rotations are retrieved from KITTI label file and then the reflected points from the objects are colored based on the height.

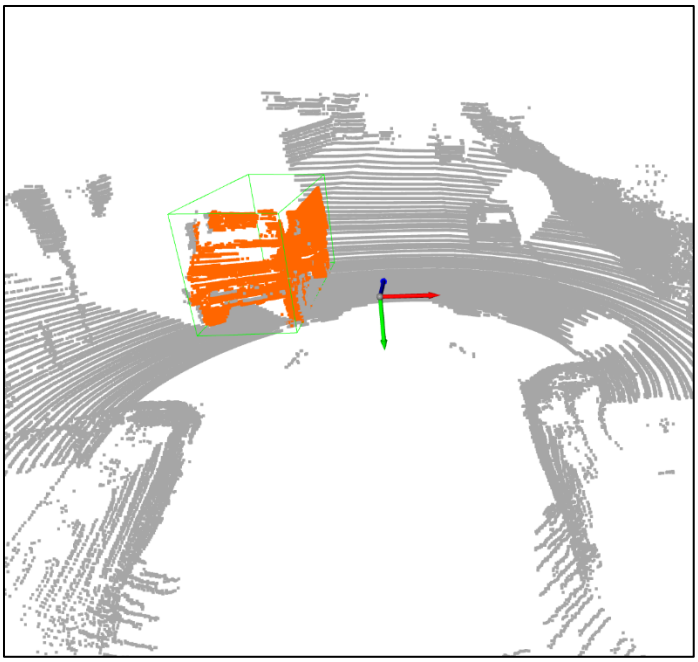
Label values	Description
h, w, l	Box height, width, and length in meters.
x, y, z	KITTI camera-frame location used as the bottom-face center.
Rotation(yaw)	Yaw angle in radians; rotates the box footprint about its vertical axis.



Construction of 2D boxes to 3D bounding boxes

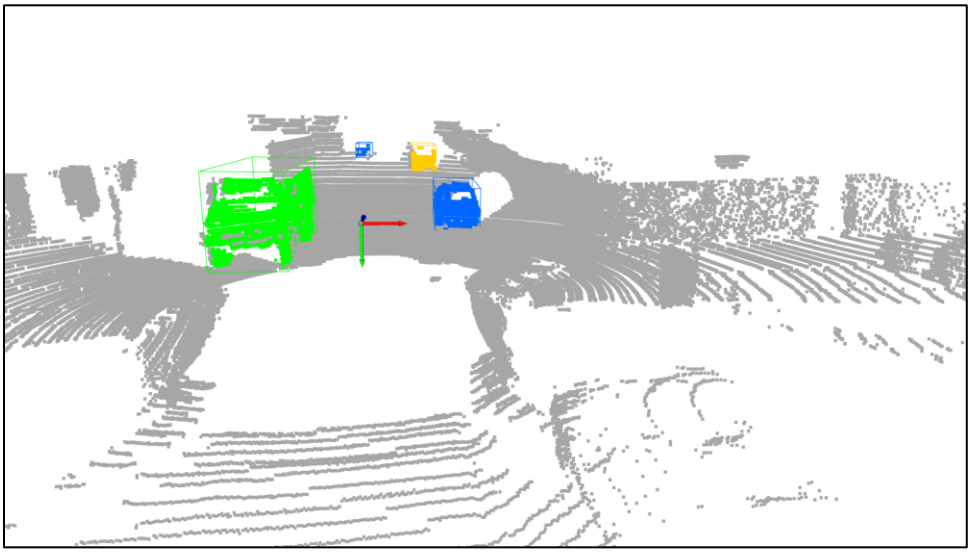
The code starts with four local footprint corners at plus or minus half the length and width. It rotates those corners by yaw, shifts them to the labelled x and z location, and duplicates the footprint at the bottom and top y values. The top is bottom_y - height because the camera y axis points downward. Connecting the corners produces the visible 3D cuboid.





Multi Object 3D Visualization

The Open3D view uses the same rectified point cloud for every supported object. Each object's label defines its box; returns inside each box receive that class color, while other returns remain grey. This lets one scan show multiple object classes at once.



Conclusion:

The project establishes the geometry and visualization foundation for LiDAR 3D detection: frame matched calibration, oriented cuboids, and point membership across multiple object classes. The next technical milestone is to generate boxes from model predictions and evaluate them against the annotations.

References:

[1] A. Mousavian, D. Anguelov, J. Flynn and J. Kosecka: 3D Bounding Box Estimation Using Deep Learning and Geometry. CVPR 2017.

[2] P.L.A.GeigerandR.Urtasun. Are we ready for autonomous driving? The KITTI vision benchmark suite. In CVPR, 2012.

[3] A. H. Lang et al., “PointPillars: Fast Encoders for Object Detection from Point Clouds,” CVPR 2019.